ISTQB® Certified Tester

Advanced Level — **Security Tester** (CT-SEC, Syllabus 2016)

CHAPTER 4

Security Testing Throughout the Software Lifecycle

Complete Study Guide

Chapter fact	Value
Syllabus sections covered	4.1 The Role of Security Testing in a Software Lifecycle · 4.2 … in Requirements · 4.3 … in Design · 4.4 … in Implementation Activities · 4.5 … in System and Acceptance Test Activities · 4.6 … in Maintenance
Learning objectives	12 examinable (AS-4.1.1 – AS-4.6.1) · K2 = 3 · K3 = 6 · K4 = 3
Recommended teaching time	225 minutes — **16 points**, 4.3 points per teaching hour
Questions in the exam	**8 of 45 questions · 16 of 80 points — the largest chapter.** K2 = 2 · **K3 = 4** · **K4 = 2**. Guaranteed: **AS-4.1.1** (K2), **4.1.2** (K3), **4.4.3** (K4), **4.5.2** (K3). Pools: 4.2.1/4.3.1 (K4) · 4.4.1/4.4.4 (K2) · 4.4.2/4.4.5 (K3) · 4.5.1/4.6.1 (K3)
Exam format	45 questions · **80 points** · **120 minutes** (150 with the 25% non-native-language allowance) · pass mark 65% = **52 points** · K2 = 1 pt · K3 = 2 pts · K4 = 3 pts
Keywords	abuse case · fuzz testing
Official sample-exam questions	**#17 – #24** (1 + 2 + 3 + 1 + 2 + 3 + 2 + 2 points)

Built exclusively from the ISTQB® Advanced Level Security Tester syllabus (2016), the official Sample Exam (questions and answers, GA March 2016) and the ISTQB® exam structure tables.

Study this single file to master Chapter 4.

Contents

Section 1 — Syllabus Explanation....4
AS-4.1.1 (K2) — Why Security Is Best Achieved Within a Lifecycle....5
AS-4.1.2 (K3) — Security-Related Activities in the Software Lifecycle....5
AS-4.2.1 (K4) — Analyzing Requirements from the Security Perspective....6
AS-4.3.1 (K4) — Analyzing a Design from the Security Perspective....7
§4.4 — Security Testing in Implementation: the frame....8
AS-4.4.1 (K2) — Security Testing During Component Testing....8
AS-4.4.2 (K3) — Security Test Design at the Component Level....9
AS-4.4.3 (K4) — Analysis of Security Tests at the Component Level....9
AS-4.4.4 (K2) — Security Testing During Component Integration Testing....10
AS-4.4.5 (K3) — Security Test Design at the Component Integration Level....10
AS-4.5.1 (K3) — Security Testing in System Testing....11
AS-4.5.2 (K3) — Security Testing in Acceptance Testing....11
AS-4.6.1 (K3) — Security Testing in Maintenance....12
Section 2 — Key Concepts....13
2.1 The storyline of Chapter 4....13
2.2 Why a lifecycle, and what happens in each phase....13
2.3 Finding deficiencies in requirements and designs (the K4 pool)....14
2.4 Component and integration testing....15
2.5 System, acceptance and maintenance testing....18
2.6 Fuzzy points clarified....19
2.7 Chapter 4 keyword glossary....19
Section 3 — Tables and Summaries....21
Section 4 — Demo Questions with Answers....23
Part A — The 8 official Chapter 4 questions....23
Part B — 24 additional syllabus-derived questions....26
Most Important Questions From This Chapter....33
Section 5 — Exam Tricks....34
5.1 Frequently tested topics (evidence-based)....34
5.2 Common distractors observed....34
5.3 Keyword recognition (map words → answers)....34
5.4 Typical traps....35
5.5 How to eliminate wrong answers....35
5.6 Scenario-solving shortcuts....35
5.7 Common candidate mistakes....35
5.8 Mnemonics that carry the whole chapter....36
5.9 Chapter 4 mind map....36
Section 6 — Revision Notes....37
6.1 One-page chapter summary....37
6.2 Key definitions to memorize....37
6.3 High-priority concepts (study first)....37
6.4 Last-minute revision sheet....37
6.5 Memorization checklist....38
6.6 Exam-day quick review....38

In Word, right-click the table of contents and choose "Update Field" → "Update entire table" to refresh the page numbers.

How to use this guide

Section 1 explains all twelve learning objectives — read it once, slowly. **Section 2** explains every key concept from the ground up — plain words, how it works, a worked example, the exam angle, and what it is easily confused with — and ends with the chapter's **fuzzy points clarified**. If the syllabus wording in Section 1 feels dense, read the matching Section 2 card first. **Section 3** holds the tables to reproduce from memory; **table 3.3 (rule → valid/invalid conditions)** and **table 3.4 (acceptance-criteria filter)** pay directly. **Section 4** is the question bank, **Section 5** exam tactics, **Section 6** is the revision sheet for the night before.

What makes Chapter 4 unusual: it is **one-fifth of the paper (16 points)** and the most hands-on chapter — four K3 test-design questions and two K4 analysis questions. **Only two of its eight questions are K2.** It is the chapter you cannot afford to be weak in.

The rule that decides most of it: test exactly what the specification says. Minimum EP sets contain **one value per class including a valid one**; condition lists contain **the valid and invalid case for every stated rule and nothing else**; acceptance criteria **restate the requirement word for word** — "logical but unspecified" criteria are wrong. The official Sample Exam uses **one password-reset requirement for two questions**.

Suggested schedule: Day 1 — §4.1–4.3 (lifecycle, requirements checklist, IEEE ten). Day 2 — §4.4 (component, CERT ten, coverage, integration). Day 3 — §4.5–4.6 and Section 3. Day 4 — Section 4 Part A, then Part B. Day 5 — Section 5. Day 6 onwards — redo the questions you missed and re-read Section 2's fuzzy points. Night before — Section 6.

Section 1 — Syllabus Explanation

Chapter 4, **Security Testing Throughout the Software Lifecycle**, carries **12 learning objectives** across six sub-chapters (4.1 Role in a Lifecycle · 4.2 Requirements · 4.3 Design · 4.4 Implementation · 4.5 System and Acceptance · 4.6 Maintenance). The Exam Structure Table allocates **8 questions** and **16 of the 80 points — the largest chapter on the paper**, one-fifth of all points. Teaching time is **225 minutes — 4.3 points per teaching hour**.

It is also the chapter with the **most K3 and K4** content: **four K3 questions and two K4s**. Only **two of its eight questions are K2**. You cannot pass CT-SEC comfortably without Chapter 4: a perfect score everywhere else is **64 points**, and a weak Chapter 4 makes 52 a tightrope.

LO ID	K	Objective (syllabus statement)	Qs	Pts
AS-4.1.1	K2	Explain why security is best achieved within a lifecycle process.	**1**	1
AS-4.1.2	**K3**	Implement the appropriate security-related activities for a given software lifecycle (e.g. iterative, sequential).	**1**	**2**
AS-4.2.1	**K4**	Analyze a given set of requirements from the security perspective to identify deficiencies.	1 of 2†	3
AS-4.3.1	**K4**	Analyze a given design document from the security perspective to identify deficiencies.	1 of 2†	3
AS-4.4.1	K2	Understand the role of security testing during component testing.	1 of 2‡	1
AS-4.4.2	**K3**	Implement component level security tests (abstract) given a defined coding specification.	1 of 2§	2
AS-4.4.3	**K4**	Analyze the results from a given component level test to determine the adequacy of code from the security perspective.	**1**	**3**
AS-4.4.4	K2	Understand the role of security testing during component integration testing.	1 of 2‡	1
AS-4.4.5	**K3**	Implement component integration security tests (abstract) given a defined system specification.	1 of 2§	2
AS-4.5.1	**K3**	Implement an end-to-end test scenario for security testing which verifies one or more given security requirements and tests a described functional process.	1 of 2¶	2
AS-4.5.2	**K3**	Demonstrate the ability to define a set of acceptance criteria for the security aspects of a given acceptance test.	**1**	**2**
AS-4.6.1	**K3**	Implement an end-to-end security retest/regression test approach based on a given scenario.	1 of 2¶	2
Chapter 4 total	—	**K2 = 2 · K3 = 4 · K4 = 2**	**8**	**16**

Pools: † 4.2.1 / 4.3.1 (K4) · ‡ 4.4.1 / 4.4.4 (K2) · § 4.4.2 / 4.4.5 (K3) · ¶ 4.5.1 / 4.6.1 (K3). Sample Exam draws: ***4.3.1*** *(Q19),* ***4.4.1*** *(Q20),* ***4.4.5*** *(Q21),* ***4.5.1*** *(Q23).*

Exam allocation you must know

Chapter 4 = **8 questions / 16 points**; Sample Exam **#17 to #24** (1 + 2 + 3 + 1 + 2 + 3 + 2 + 2 points). The two guaranteed K3s (**AS-4.1.2, 4.5.2**) and the guaranteed K4 (**AS-4.4.3**) are **7 points** on their own.

The one thing that decides this chapter

Test exactly what the specification says — no more, no less. The K3 questions in this chapter are **test-design exercises**: minimum equivalence classes for an input (Q22), the minimum condition set for a requirement (Q23), the acceptance criteria a requirement actually supports (Q24). The right answer **covers both valid and invalid behaviour of the stated rules**, does **not** add attack tests the question did not ask for, and does **not** invent rules the requirement does not state ("tempting and logical, but not specified").

AS-4.1.1 (K2) — Why Security Is Best Achieved Within a Lifecycle

Original syllabus statement

Explain why security is best achieved within a lifecycle process.

Security is not tested or patched into an already-built application. Rather, it is achieved through security-oriented design and verification throughout the process of construction. The concepts apply to **any lifecycle — sequential or iterative** [IEEE 12207]. The syllabus gives **four reasons** for integrating security testing into the lifecycle:

Reason	Syllabus example
1. A prescribed time for security-related activities	When capturing needs, the analyst asks: **which levels of security access are needed? any assets needing special defenses? how "open" is the application? what are the security risks?** During **coding**, the developer has **the best opportunity to apply secure coding practices** against **SQL injection and buffer overflow** — finding them later is **difficult and costly** because many components may need the same fix.
2. Checkpoints for review	**Security requirements or user stories reviewed** for the security aspects of user needs; **code changes reviewed to detect malicious coding by internal employees or contractors**.
3. Checkpoints for testing	In development, **component tests documented and performed to verify secure coding practices** have been followed.
4. Entry and exit criteria throughout the project	**No component accepted into an integrated test environment until all security-related activities (development and testing) are complete** — especially important later, when a vulnerability **could impact the entire system**.

How the official exam used this objective

Sample Q17: at what point should there be checking that **proper secure coding practices have been followed**? **Component testing** — official rationale: **the checking should be done as soon as the code is written.** The syllabus: in development, **component tests should be documented and performed to verify that secure coding practices have been followed**.

AS-4.1.2 (K3) — Security-Related Activities in the Software Lifecycle

Original syllabus statement

Implement the appropriate security-related activities for a given software lifecycle (e.g. iterative, sequential).

These activities are **performed alongside other project activities, not in a separate lifecycle**.

Objectives and activities by phase

Phase	Objectives	Activities
Requirements	**Understand and identify security needs from all perspectives inside and outside the organization** (e.g. the customer's need for private data to stay secure) · **document them in a detailed and unambiguous way** so implementation and testing are **traceable** and requirements can be **verified and validated**. Requirements may go beyond user needs: **regulatory, technical, business**.	**Define all impacted and knowledgeable people** · gather needs from each group via **interviews, workshops**, etc. (can be done with other elicitation) · **document so they can be reviewed and traced · review for correctness, completeness, understandability and non-ambiguity**.
Design	**Create a design that meets the stated security requirements** — requirements express needs; design translates them into a workable solution.	**Analyze** the documented requirements · arrive at **the most workable approach to develop securely** · **document** the design appropriately for the lifecycle (whiteboard in iterative, models in others).
Implementation	**Translate requirements and design into secure code**	**Create code meeting security requirements** ·

Phase	Objectives	Activities
(coding)	meeting functional needs · implement other **procedures or technology** (firewalls, tokens…).	**component testing** for correctness, efficiency and security · **component reviews** to visually inspect the same.
System testing	**End-to-end test** of the full system (**hardware, software, data, people, procedures**) after integration · test that **security requirements are implemented correctly from a system perspective**. Iterative models can system-test **much more frequently**.	Perform security tests **in some approximation of the final target environment** — a **transition from the development environment**.
Acceptance testing	**Users or their agents perform security testing against security-related acceptance criteria** — often focused on **functional security controls and processes**.	**Install into the operational environment · perform tests based on acceptance criteria · determine acceptance** from results.
Maintenance	Corrective, adaptive or perfective changes; **confirmation testing** of fixes and **regression testing** of core functionality to **ensure no new vulnerabilities** and **verify existing defenses still effective**.	Covers **upgrades (OS, databases), coding changes, data conversions, platform migrations — treated with the same care as original development**.

System and acceptance testing are essentially black-box (stimulus-response). Component and integration testing are complementary — they **consider and exploit the internal architecture of components and their interactions**.

How the official exam used this objective (K3, 2 points)

Sample Q18: a safety-critical medical system — when and how should security requirements be documented? **In a detailed and unambiguous way in the requirements documents during the requirements phase.** Rejected: **not documented formally** to hide them (they should be documented, but **protected from those who don't need to know**); **during design** once the code approach is known (they may be **refined** in design, but are **captured in requirements**); **only functional access and availability** requirements (security requirements also include **secure coding practices, etc.**).

AS-4.2.1 (K4) — Analyzing Requirements from the Security Perspective

Original syllabus statement

Analyze a given set of requirements from the security perspective to identify deficiencies.

Five general truths about requirements set the context:

1. Many organizations **struggle to write basic requirements** that are **clear, unambiguous, complete, correct and testable**.
2. Requirements are **highly subject to change**, so **maintaining** them is a challenge.
3. **Special skills** are needed to understand user, compliance and technical needs before documenting them.
4. Requirements **contain gaps and errors**, so **both verification and validation** are needed.
5. Requirements **should contain quality characteristics** — security, performance, usability — but **these are often overlooked in favor of functionality only**.

An effective technique is a checklist. The syllabus's security starting point has **six headings**:

The security requirements checklist

Heading	Checklist questions
Privacy needs	All **user groups and their data privacy needs** identified and documented? All **data types** impacted identified with privacy needs? **User access rights** identified and defined?
Compliance needs (to security policies)	All **relevant security policies** identified and documented? Any **exceptions to policies** identified and documented?
Common vulnerabilities	All **common and known vulnerabilities for the feature** identified as **known risks**? (They change over time, are defined as risks at requirements time, and **become the basis for security tests**.)

Heading	Checklist questions
Testability	Written so **security tests and other tests can be written** from it? **Ambiguous terms** — e.g. **"processing must be secure"**, **"access is only granted to authorized personnel"** — identified and made **specific and testable**?
Usability	Trade-off: a login **so confusing customers give up and go elsewhere**. Appropriate level of security process **for the function**? Procedures **clear and understandable**? **Remedies for legitimate users** who have access problems?
Performance	Trade-off: **high levels of encryption can slow performance**. Appropriate level of **security efficiency** for the function?

K4 method for a requirements question

Run the six headings over the requirement text. Typical deficiencies:

Privacy — a data type or user group with no stated privacy need. **Compliance** — no reference to the governing policy, or an unstated exception. **Vulnerabilities** — an input feature with no stated injection or overflow risk. **Testability** — vague words ("secure", "authorized", "appropriate"). **Usability** — no remedy for a locked-out legitimate user. **Performance** — no bound on the cost of a security mechanism.

The best answer usually identifies the deficiency **that blocks testing** (ambiguity) or the one **with the highest risk** to the asset in the stem.

AS-4.3.1 (K4) — Analyzing a Design from the Security Perspective

Original syllabus statement

Analyze a given design document from the security perspective to identify deficiencies.

Security-degrading design practices should be identified and avoided. Testing contributes by **recognizing designs likely to be vulnerable to compromise** and **directing design toward strong, identifiable security properties**. The **IEEE Computer Society Center for Secure Design** recommends ten key approaches:

IEEE Center for Secure Design — ten design approaches

#	Approach	Deficiency it reveals in a design
1	**Earn or give, but never assume, trust**	Client-side checks trusted by the server; implicit trust between components
2	**Use an authentication mechanism that cannot be bypassed or tampered with**	Alternate paths (deep links, APIs) that skip login
3	**Authorize after you authenticate**	Authorization checked once at login and never again; URLs usable by anyone
4	**Strictly separate data and control instructions; never process control instructions from untrusted sources**	Injection — data interpreted as commands
5	**Define an approach that ensures all data are explicitly validated**	Unvalidated inputs, inconsistent validation
6	**Use cryptography correctly**	Home-grown crypto, weak keys, wrong modes
7	**Identify sensitive data and how it should be handled**	Sensitive data unclassified, logged or cached
8	**Always consider the users**	Security so hard to use that it is bypassed
9	**Understand how integrating external components changes your attack surface**	Third-party libraries and services added without review
10	**Be flexible when considering future changes to objects and actors**	Designs that break when roles or components change

How the official exam used this objective (K4, 3 points)

Sample Q19: in production, **copying a URL from an authorized session into an unauthorized session** lets the attacker continue with the authorized user's rights — and change the admin password. The fix: **check the session ID and the user ID any time a URL is used**. Realistic concern? **It will fix the problem, but performance may be adversely affected** — official rationale: **this level of checking will slow down the system because it will have to check on each screen change.**

Rejected: it **will** fix the problem (so not a and d); **no impact on usability** (unless you are the hacker). Reasoning link: the defect violated **"authorize after you authenticate"**; the fix restores it, and the syllabus's requirements checklist names the **security/performance trade-off**.

§4.4 — Security Testing in Implementation: the frame

Security testing **begins at the lowest level of implementation**, exercising separate components that will be assembled into the system. **After the static evaluation** of components, testing adds **appraisal of dynamic behavior in response to both valid and invalid inputs**.

AS-4.4.1 (K2) — Security Testing During Component Testing

Original syllabus statement

Understand the role of security testing during component testing.

White-box / glass-box considerations (§4.4.1.1)

- **Static testing** — the whole range of **inspection, walkthrough, audit, technical review** — has already been noted.
- **White-box (structural)** tests are designed **with visibility into design or implementation**; **black-box** (functional and non-functional) is **stimulus-response** without structural information.
- White-box testing can **target specific controls implemented within the module** and determine their effectiveness; visibility also lets **coverage be measured** — % statements, % decision outcomes, % logic paths.
- Structural security testing may use **automated static analysis tools and security scanning tools**.
- **Fuzz testing** discovers vulnerabilities **by inputting massive amounts of random data, called fuzz**, to the component or system. **White-box fuzzing** on small blocks (functions, classes) **may get usable results in much less time than black-box fuzzing**; white-box fuzz tools detect **memory corruption, buffer overflow, etc. by instrumenting the code**.

Vulnerabilities identifiable and fixable during structural testing

Vulnerability	Note
Memory buffer overflows	Also found by dynamic memory tools (Chapter 3)
Malicious code inserted by an internal employee or contractor	Why code changes are reviewed (§4.1.1)
"Backdoor" access	**Access via an undocumented interface only known to the developer**, intentionally implemented **to bypass normal security controls**

Functional security testing considerations (§4.4.1.2)

Adequacy at any level is determined by confirming satisfaction of specified security requirements — in addition to observing responses to **stresses not explicitly specified** in requirements and risk assessments. **Creativity is required** because testers are **probing for what developers have overlooked**.

How the official exam used this objective

Sample Q20: why review **compiler warnings** during component testing? **Because they indicate potential issues that should be**

investigated — they **could lead to security gaps**. Rejected: they are problems that **must** be fixed (warnings **do not necessarily require a fix**); functional defects or maintainability (true perhaps, but **not related to security testing**). "**Heed compiler warnings**" is item 2 of the CERT list in §4.4.2.

AS-4.4.2 (K3) — Security Test Design at the Component Level

Original syllabus statement

Implement component level security tests (abstract) given a defined coding specification.

One set of high-level coding best practices is **CERT's "Top 10 Secure Coding Practices"**: tests for any component should include **assessment of possible violations** of these practices.

CERT Top 10 Secure Coding Practices — and an abstract component test for each

#	Practice	Abstract component-level test
1	**Validate input**	Supply out-of-range, wrong-type, over-length and metacharacter inputs; expect rejection
2	**Heed compiler warnings**	Review warnings at the highest level; investigate each as a potential security issue
3	**Architect and design for security policies**	Check the component enforces the policy it is responsible for (e.g. role check present)
4	**Keep it simple**	Flag unnecessary complexity in security-relevant code (high cyclomatic complexity)
5	**Default deny**	Request an action not explicitly permitted; expect denial
6	**Adhere to the principle of least privilege**	Verify the component runs and calls with the minimum rights needed
7	**Sanitize data sent to other systems**	Send data containing control characters to downstream calls; verify encoding/escaping
8	**Practice defense in depth**	Disable one control; verify another still prevents the attack
9	**Use effective quality assurance techniques**	Confirm static analysis, reviews, fuzzing were applied
10	**Adopt a secure coding standard**	Check the code against the adopted standard's rules

The third column is illustrative; the syllabus lists the ten practices and says tests should assess violations of them.

Focus: tests against checklists should be based on **a well-documented risk analysis incorporating realistic threat modeling — focus on the most crucial requirements in terms of the probability of attack and the consequences of compromise**.

AS-4.4.3 (K4) — Analysis of Security Tests at the Component Level

Original syllabus statement

Analyze the results from a given component level test to determine the adequacy of code from the security perspective.

One key measure of adequacy is test coverage, and coverage measures **derive from the nature of the testing performed**:

Four families of coverage

Testing type	What it gives assurance of	Coverage measures
Requirements-based (black-box)	The system **satisfies its requirements as specified**, without regard for implementation	**% of requirements tested · % of specified use/abuse cases tested · % of critical functions, scenarios, or mission threads tested**

Testing type	What it gives assurance of	Coverage measures
Data-driven	Behavior across a **range and combination of input data**, with **as few values as possible**: divide the data space into **equivalence classes** and select **one representative from each class**	**Equivalence class coverage · pairwise · N-wise** coverage criteria
Model-based	Coverage **in terms of a chosen modeling notation**	**Pre-post notation:** cause-effect coverage, coverage of all **disjuncts in the postcondition**. **Algebraic:** coverage of the **axioms**. **Transition-based** graphs: **% nodes (states), % transitions, % transition-pairs, % cycles**
Structural	Based on **visibility into the actual implementation**	**% packages, classes, methods, decisions or lines of executable code** executed — **lines = statement coverage**

Control-flow criteria

- **Cyclomatic complexity** = **how many different independent paths exist** through an element; visualized as a **control flow graph** (nodes = decision points, arcs = paths).
- **Path coverage** — all entry-to-exit paths — is **the strongest** control-flow criterion, but **exhaustive path testing is generally not feasible because of loops**.
- Less stringent alternatives: **critical path coverage** (selected logical paths considered critical) and **branch coverage** (**% of decision outcomes** exercised).

How the official exam used this objective (K4, 3 points)

Sample Q22: SQL injection tests for a field accepting **up to 5 alphanumeric characters**, using **equivalence partitioning** for the **minimum** set. Answer: **' and ab123** — **one test for SQL injection and one for a valid input**.

Rejected: **bbbbb, 12345, '** and **%, ', @, ab123** have **more than the minimum**; **' alone** is **not enough because it doesn't test the valid input**. The rationale adds that more SQL-injection characters would be advisable in practice, **but the question asks to apply EP and get the minimum**. Data-driven rule: **one representative per class** — here, two classes: valid alphanumeric and SQL metacharacter.

AS-4.4.4 (K2) — Security Testing During Component Integration Testing

Original syllabus statement

Understand the role of security testing during component integration testing.

- As components integrate into subsystems and the full system, **breach possibilities are not simply the summation of the vulnerabilities in each component** — **new attack vectors become possible** through interactions **between components and with larger system and organizational elements**.
- Conversely, **some interactions might mitigate or block** possible breach sequences.
- Testers must again be **creative** in looking for what developers overlooked.
- Integration testing shows **the complexity of a system design and the stability of its behavior**; the **integration approach (top-down or bottom-up)** can affect **the timing of revealing security concerns** or the need for **additional security-specific tests**.

AS-4.4.5 (K3) — Security Test Design at the Component Integration Level

Original syllabus statement

Implement component integration security tests (abstract) given a defined system specification.

As with component testing, integration tests should be designed on **well-documented risk analysis incorporating realistic threat modeling**. **Scaffolding (stubs and drivers)** may be needed to test less-than-complete paths. As more real components are added, **scaffolding is incrementally removed**, allowing **fuller assessment of functionality** and revealing **new paths to vulnerabilities**.

How the official exam used this objective (K3, 2 points)

Sample Q21: 20 components, each extensively security-tested; now integration. Approach? **New vulnerabilities are likely present in the integrated components and in the larger system and infrastructure that is now testable, so testing should expand to include these new areas.** Rejected: integration = **summation** of component vulnerabilities (a); testing **only interfaces plus retest** (b — too narrow); risks **reduced** by integration so only integration points tested (d — risks are **likely increased**).

AS-4.5.1 (K3) — Security Testing in System Testing

Original syllabus statement

Implement an end-to-end test scenario for security testing which verifies one or more given security requirements and tests a described functional process.

- System testing is **the first end-to-end exercising of the fully integrated components**. Usually in a **development environment**, it should reveal **emergent properties** not observable before integration.
- Security requirements are **typically considered in conjunction with one or more functional requirements**: **"In the process of doing x the system should not allow y to happen."** As functional tests run, **probe for ways security constraints might be violated**.
- Functional requirements **address the imperatives**. **Use cases, abuse cases, process models and state transition models** describe **procedures** from which **end-to-end security scenarios** are defined.

How the official exam used this objective (K3, 2 points)

Sample Q23: the **password-reset requirement** (two of three security questions; emailed link; link disabled after 1 hour; two requests without reset, then helpdesk; other errors lock the ID). Minimum test conditions for **functional security**? **Invalid user; valid user; 2 correct answers; 2 incorrect answers; good email; bad email; reset with good password; reset with bad password; link good; link expired; two requests without reset; three requests without reset.**

Rejected: **valid conditions only** (b); **error conditions only** (c); and (d), which **expands into attack testing** (buffer overflow, SQLi, XSS) beyond the functional security asked for. **Each rule in the requirement yields a valid and an invalid condition.**

AS-4.5.2 (K3) — Security Testing in Acceptance Testing

Original syllabus statement

Demonstrate the ability to define a set of acceptance criteria for the security aspects of a given acceptance test.

- Acceptance testing differs from system testing by being performed **in a realistic operational environment**, if not the actual one — allowing evaluation of **performance and other behaviors through external interfaces**, and placing the system **where external threat agents will seek weaknesses day to day**.
- It should **validate that initial project goals have been delivered**, by designing and performing tests that **acceptance criteria are met**. **Security needs should be documented in the acceptance criteria.**
- **The best time to define acceptance criteria is before system development or purchase**, so supplier and acquirer (even in the same organization) reach an initial understanding. Criteria **commonly change or emerge**; **analyze their impact on security testing**.
- Criteria may be **global** — applied to all functions: **user authentication, user rights, encryption levels, audit trails** — or **specific** — e.g. **payments above an amount require two people to approve**.

How the official exam used this objective (K3, 2 points)

Sample Q24, same password-reset requirement: valid acceptance criteria are **1, 3, 5, 6, 8** — reset succeeds under the stated conditions (1); **more than two requests without a reset → ID locked** (3); **more than two security questions missed → ID locked** (5); **email error → ID locked** (6); **reset password works for login** (8).

Why not the others: **2** omits "without a reset" (3 is the correct form); **4** says "error" where the requirement says **locked** (5 is correct); **7** (prompting with password rules) is **tempting and logical, but not specified in the requirement**. **Acceptance criteria restate the requirement precisely — they do not improve it.**

AS-4.6.1 (K3) — Security Testing in Maintenance

Original syllabus statement

Implement an end-to-end security retest/regression test approach based on a given scenario.

- **Regression testing** confirms previously acceptable behaviors remain after modifications. In the negative aspects of security testing, that means checking **the system continues to successfully resist attempts to defeat established controls**.
- **Enhancements to usability or efficiency are especially prone to sacrificing security controls.**
- Security regression tests focus on **confirming all security requirements** and **testing for new vulnerabilities** introduced by maintenance.
- Regression suites built from **individual function tests are often insufficient** for security. **End-to-end regression scenarios are more robust** and give **higher confidence that complete transactions can be performed securely**.
- Define **a set of security test scenarios run each time a change is made** — changes include **hardware, configuration files, OS, DBMS, networking and software**; **any** of these can cause regression defects with security impact.

The syllabus's example end-to-end security regression scenarios

Scenario
Users can **log in and complete a purchase securely without compromising personal information**.
Users can **only perform actions defined in their user rights and privileges** — a payroll user may **add a new employee** but **not access their banking information**.

Section 2 — Key Concepts

Section 1 gives the syllabus wording objective by objective. This section explains what that wording means. Each card builds one idea in order: plain words first, then how it works, an example, how the exam uses it, and what it is easily confused with. Chapter 4 is one-fifth of the exam, and most of its questions are test-design exercises, so the worked examples here matter most.

2.1 The storyline of Chapter 4

Chapter 4 follows a system **from its first requirement to its last patch** and asks what security testing does at each stop. **Requirements**: are the security needs written down, clear and testable? **Design**: does the design follow secure-design principles? **Components**: does each piece of code follow secure coding practice, and how much of it did we test? **Integration**: what new attack paths appear when pieces connect? **System**: does the whole thing resist misuse end to end? **Acceptance**: does it meet the security acceptance criteria in a realistic environment? **Maintenance**: after every change, does it **still** resist attack?

One rule runs through all the K3 questions: **test exactly what the specification says.** Cover the valid and the invalid behaviour of **every stated rule**. Do not add attack tests the question did not ask for, and do not invent rules the requirement does not contain.

2.2 Why a lifecycle, and what happens in each phase

Security is built in, not tested in (AS-4.1.1, guaranteed K2)	
In plain words	You cannot take a finished, insecure application and **test or patch** security into it. Security comes from **secure design and verification at every step of construction**. That is true in **any** lifecycle, sequential or iterative.
How it works	**1 A prescribed time for security activities.** The analyst asks the security questions while capturing needs (which access levels? which assets need special defenses? how open is the application? what are the risks?). The **developer** has the **best chance to apply secure coding** (against SQL injection and buffer overflow) **while writing the code**. **2 Checkpoints for review**: security requirements and user stories are reviewed, and **code changes are reviewed for malicious code** inserted by employees or contractors. **3 Checkpoints for testing**: **component tests** are documented and run to verify **secure coding practices were followed**. **4 Entry and exit criteria**: **no component enters the integrated test environment until its security activities are complete**. A vulnerability found later could affect the **whole** system.
Example	A team finds the same SQL injection flaw in 40 components during system testing. Fixing it now means touching 40 components and retesting them all. Had component testing checked secure coding **when each component was written**, it would have been one fix per component, made by the developer who had just written the code.
Exam angle	Official Q17: when should you check that secure coding practices were followed? → **Component testing**, **as soon as the code is written**.
Don't confuse	**"Lifecycle"** here means **any** development model. An option saying lifecycle security only applies to sequential (or only to Agile) projects is wrong.

Security activities phase by phase (AS-4.1.2, guaranteed K3)	
In plain words	Each phase has a **security objective** and **activities** that run **alongside** the normal project work, not in a separate security project.

Security activities phase by phase (AS-4.1.2, guaranteed K3)	
How it works	**Requirements**: find security needs **from all perspectives, inside and outside** the organization (including regulatory, technical and business needs), and **document them in a detailed and unambiguous way** so they can be **traced, verified and validated**. Review them for correctness, completeness, understandability and non-ambiguity. **Design**: produce a design that **meets the stated security requirements**. Document it in the way the lifecycle uses (a whiteboard in iterative work, models elsewhere). **Implementation**: write **secure code**, run **component tests** and **component reviews**, and implement other procedures and technology (firewalls, tokens). **System testing**: an **end-to-end** test of hardware, software, data, people and procedures, run **in some approximation of the final target environment**. Iterative models can do this much more often. **Acceptance testing**: **users or their agents** test against **security acceptance criteria** in the **operational environment**, often focusing on functional security controls and processes. **Maintenance**: **confirmation testing** of fixes and **regression testing**, to make sure there are no new vulnerabilities and that **existing defenses still work**. Upgrades, data conversions and migrations get **the same care as original development**.
Example	A safety-critical infusion-pump system: during requirements the team writes “Only a clinician with role ‘Prescriber’ may change a dose above 10 mg/h; every change is logged with user and time.” It is detailed, unambiguous and testable, so design, code and tests can all trace back to it.
Exam angle	Official Q18: when and how should security requirements be documented? → **in a detailed and unambiguous way in the requirements documents, during the requirements phase.** They may be **refined** in design, but they are **captured** in requirements. They should be documented **and** protected from people who do not need to know them, not left undocumented.
Don’t confuse	**System and acceptance testing are essentially black-box** (stimulus and response). **Component and integration testing** use knowledge of the internals.

2.3 Finding deficiencies in requirements and designs (the K4 pool)

Analyzing requirements with the security checklist (AS-4.2.1)	
In plain words	Requirements are often unclear, keep changing, contain gaps, and **forget quality characteristics such as security** in favour of functionality. The syllabus’s tool for finding the gaps is a **checklist with six headings**.
How it works	**Privacy needs**: are all user groups, data types and access rights identified, with their privacy needs? **Compliance needs**: are the relevant security policies (and any **exceptions** to them) identified? **Common vulnerabilities**: are the known vulnerabilities for this feature listed as **known risks**? They become the **basis for security tests**. **Testability**: could you write a test from it? Vague words such as **“processing must be secure”** or **“only authorized personnel”** must be made specific. **Usability**: is the security effort right for the function? Are the procedures clear? Is there a **remedy for legitimate users** who get locked out? **Performance**: is the cost of the security mechanism acceptable? **Strong encryption can slow the system.**
Example	Requirement: “Customer statements must be stored securely and only authorized staff may view them.” Checklist results: **testability**: “securely” and “authorized” are undefined (which encryption? which roles?). **Privacy**: which data fields are personal? **Usability**: what happens when a staff member needs urgent access? **Performance**: no bound on the decryption time for a 5-year history. **Common vulnerabilities**: the statement viewer takes an account number in the URL, but no risk of changing that number to see someone else’s statement is listed.
Exam angle	In a K4 requirements question, run all six headings. The best answer usually names the deficiency that **blocks testing** (ambiguity) or the one with the **highest risk** to the asset in the stem.

Analyzing requirements with the security checklist (AS-4.2.1)	
Don't confuse	**Usability and performance** are on a **security** checklist because security that is too hard or too slow gets **bypassed or switched off**. Do not discard these options as off-topic.

Analyzing a design: the IEEE Center for Secure Design's ten principles (AS-4.3.1)	
In plain words	Testers help by **spotting designs that are likely to be vulnerable** before code is written. The IEEE's ten principles are the checklist. Each principle, when it is broken, gives a recognisable deficiency.
How it works	**1 Earn or give, but never assume, trust**: the server trusts checks done in the browser. **2 Authentication that cannot be bypassed or tampered with**: a deep link or API skips the login. **3 Authorize after you authenticate**: rights are checked once at login and never again. **4 Separate data from control instructions**: user data gets executed (injection). **5 Explicitly validate all data**: some inputs are not validated, or are validated inconsistently. **6 Use cryptography correctly**: home-made crypto, weak keys, wrong modes. **7 Identify sensitive data and how to handle it**: sensitive data is logged, cached or left unclassified. **8 Always consider the users**: security so awkward that people bypass it. **9 Know how external components change your attack surface**: third-party libraries are added without review. **10 Be flexible about future changes to objects and actors**: the design breaks when roles or components change.
Example	Official Q19: in production, someone **copies a URL from an authorized session into an unauthorized one** and carries on with the authorized user's rights. This breaks **principle 3**. The fix is to **check session ID and user ID every time a URL is used**. That fixes it, but adds a check on **every screen change**, so **performance may suffer**.
Exam angle	In Q19 the realistic concern is **"it will fix the problem, but performance may be adversely affected."** This links to the requirements checklist's **security/performance trade-off**. Options saying it will **not** fix the problem, or that it harms **usability**, are wrong.
Don't confuse	Principle **2** (cannot **bypass** authentication) ≠ principle **3** (keep **authorizing** after authentication). A URL that works without logging in breaks 2. A URL that works for the **wrong** logged-in user breaks 3.

2.4 Component and integration testing

Security in component testing: static first, then dynamic (AS-4.4.1)	
In plain words	Security testing starts at the **smallest pieces of code**. First you **examine** the code without running it (static testing). Then you **run** it with both **valid and invalid** inputs and watch how it behaves (dynamic testing).
How it works	**Static**: inspections, walkthroughs, audits, technical reviews, and **static analysis and security-scanning tools**. **White-box (structural) tests** can target the **specific controls inside the module** and **measure coverage** (statements, decisions, paths). **Black-box** tests use only stimulus and response. **Fuzz testing**: feed **massive amounts of random data ("fuzz")** to find crashes and weaknesses. **White-box fuzzing** on small units (functions, classes) can give useful results **much faster** than black-box fuzzing, and its tools **instrument the code** to detect **memory corruption and buffer overflows**. **Found best here: buffer overflows**, **malicious code** inserted by an insider, and **backdoors** (an **undocumented interface**, known only to the developer, that **bypasses normal security controls**). **Adequacy** = the specified security requirements are met, **plus** the response to stresses that were **not** specified. **Creativity** is needed, because you are looking for what the developers overlooked.

Security in component testing: static first, then dynamic (AS-4.4.1)	
Example	Reviewing a login component, a reviewer notices: if user name = “svcmaint” and password = “x!9”, skip the password check. Nothing in the specification mentions it. That is a **backdoor**, and only structural review (or very lucky black-box testing) would find it.
Exam angle	Official Q20: why review **compiler warnings**? → **they indicate potential issues that should be investigated**, because they could lead to security gaps. Reject “they **must** be fixed”: warnings do **not necessarily** require a fix.
Don’t confuse	Fuzzing is **random** input in **large volume**. It is not the same as carefully chosen equivalence-class values.

Designing component tests from CERT’s Top 10 secure coding practices (AS-4.4.2, K3 pool)	
In plain words	Given a coding specification, your component tests should look for **violations of good secure-coding practice**. The syllabus uses CERT’s ten practices as that checklist, and you turn each practice into a concrete check.
How it works	**1 Validate input**: send out-of-range, wrong-type, over-long and metacharacter values → expect rejection. **2 Heed compiler warnings**: compile at the highest warning level and investigate each warning. **3 Architect and design for security policies**: check the component enforces its part of the policy (for example, a role check). **4 Keep it simple**: flag unnecessary complexity in security-relevant code. **5 Default deny**: ask for something not explicitly allowed → expect denial. **6 Least privilege**: the component runs and calls others with the **minimum** rights it needs. **7 Sanitize data sent to other systems**: send control characters downstream and check they are encoded or escaped. **8 Defense in depth**: disable one control → another still stops the attack. **9 Effective QA techniques**: static analysis, reviews and fuzzing were really applied. **10 Adopt a secure coding standard**: check the code against it.
Example	Spec: “The export function writes the customer list to a CSV for the reporting service.” Abstract tests: input validation on the date range (1); a user without the Reports role gets **denied** by default (5, 3); the function runs under a read-only database account (6); a customer name beginning with “=” is **escaped** in the CSV so the spreadsheet does not execute it (7).
Exam angle	Focus your tests on a **documented risk analysis with realistic threat modeling**: the requirements with the **highest probability of attack and the worst consequences** come first.
Don’t confuse	The specific test in each row above is **illustrative**. The syllabus lists the ten practices and says tests should assess violations of them.

Coverage: how you know you tested enough (AS-4.4.3, guaranteed K4)	
In plain words	**Coverage** is the share of something that your tests exercised. **Which** something depends on the kind of testing you did. The syllabus groups coverage measures into **four families**, and you must know which measure belongs to which family.

Coverage: how you know you tested enough (AS-4.4.3, guaranteed K4)	
How it works	**Requirements-based** (black-box): % of **requirements**, % of **use and abuse cases**, % of **critical functions, scenarios or mission threads** tested. **Data-driven**: split the input space into **equivalence classes** and pick **one representative from each class**. Measures: **equivalence-class coverage**, **pairwise** and **N-wise** combinations. **Model-based**: coverage of a **model**. For **state-transition** graphs: % **states**, % **transitions**, % **transition pairs**, % **cycles**. For **pre/post-condition** notation: cause-effect coverage and coverage of every **disjunct** of the postcondition. For **algebraic** specifications: coverage of the **axioms**. **Structural**: % of **packages, classes, methods, decisions or lines** executed. **Lines = statement coverage. Decisions = branch coverage.** **Control flow: cyclomatic complexity** = the number of **independent paths**. **Path coverage** (every entry-to-exit path) is the **strongest** criterion but is usually **impossible because of loops**. Weaker alternatives: **critical-path** coverage and **branch** coverage.
Example	Official Q22: a field accepts **up to 5 alphanumeric characters**. Using **equivalence partitioning**, what is the **minimum** set of SQL-injection tests? **Step 1, find the classes the question is about:** valid input (alphanumeric, 1–5 characters) and SQL metacharacter input (a single quote). **Step 2, take one value per class:** a single quote, and "ab123". **Answer: those two values.** "bbbbb, 12345, quote" and "%, quote, @, ab123" are **more than the minimum**. A single quote **on its own** is **not enough**, because the **valid** class is untested.
Exam angle	Read the measure in the stem and name its family (Section 3, table 3.2). For EP minimum questions: **one value per class, and always include a valid class.**
Don't confuse	**Statement** coverage (lines) ≠ **branch** coverage (decision outcomes) ≠ **path** coverage (whole routes). Each is stronger than the one before. In practice you would test **many** injection characters. The exam asks for the EP **minimum**, so answer the question asked.

Integration testing: the whole is not the sum of the parts (AS-4.4.4 and AS-4.4.5)	
In plain words	Joining secure components does **not** give a system that is just "the components' vulnerabilities added up". **Connections create new attack paths.** Occasionally a connection **blocks** one.
How it works	**New attack vectors** come from interactions **between components** and with the **larger system and organization** (networks, other systems, people). Some interactions **mitigate or block** a breach sequence. **Integration order (top-down or bottom-up)** changes **when** security problems become visible, and may require **extra** security tests. **Scaffolding** (stubs and drivers) lets you test incomplete paths. As real components replace it, the scaffolding is **removed step by step**, and **new paths to vulnerabilities appear**. Design integration tests from a **documented risk analysis with realistic threat modeling**, as at component level.
Example	Component A (a file-upload service) is secure: it rejects executable files. Component B (a thumbnail generator) is secure: it only reads images. Integrated, A stores the upload where B reads it, and B's image library has an exploitable parsing flaw. A crafted "image" now reaches B. Neither component had this vulnerability alone.
Exam angle	Official Q21: 20 components, each well tested, are now being integrated → **expect new vulnerabilities in the integrated components and in the larger system and infrastructure that can now be tested; expand testing to cover them.** Reject "sum of component vulnerabilities", "test only interfaces", and "integration reduces risk".
Don't confuse	Integration testing is **not** just interface testing. The syllabus wants the **larger system and infrastructure** included.

2.5 System, acceptance and maintenance testing

End-to-end system test scenarios: turning a requirement into test conditions (AS-4.5.1)	
In plain words	System testing is the **first time the whole integrated system runs end to end**. It reveals **emergent properties** that no single component shows. Security requirements are usually stated **inside** functional processes: **"while doing x, the system must not allow y."** For each rule in the requirement you test **one valid and one invalid** condition.
How it works	Build scenarios from **use cases, abuse cases, process models and state-transition models**. While the functional steps run, **probe for ways the security constraint could be broken**. **Method for a requirement:** list every rule it states → write a **valid** and an **invalid** condition for each → stop. **Do not add** unrelated attack tests unless the question asks for them.
Example	The password-reset requirement (official Q23): the user must exist; must answer **2 of 3** security questions; gets a link by **email**; the link **expires after 1 hour**; **two** requests without a reset are allowed, then the helpdesk; other errors lock the ID. Conditions: **invalid user / valid user · 2 correct answers / 2 incorrect answers · good email / bad email · reset with good password / bad password · link good / link expired · two requests without reset / three requests without reset.**
Exam angle	Official Q23: the right option has **both valid and invalid** conditions for **every** rule. Reject **valid-only** options, **error-only** options, and the option that **adds buffer overflow, SQL injection and XSS**, because the question asked for **functional security**, not attack testing.
Don't confuse	**"Minimum set"** in Chapter 4 means **complete for the stated rules, with nothing extra**. It does not mean "as few tests as possible".

Security acceptance criteria: restate the requirement exactly (AS-4.5.2, guaranteed K3)	
In plain words	Acceptance testing checks, in a **realistic operational environment**, that what was promised was delivered. **Security needs must be written into the acceptance criteria**, and the criteria must say **exactly what the requirement says**: no more, no less, no rewording.
How it works	**Best time to define acceptance criteria: before** development or purchase, so the supplier and the acquirer agree early. Criteria often **change or emerge**, so analyze the impact of each change on security testing. **Global criteria** apply to every function: **user authentication, user rights, encryption levels, audit trails**. **Specific criteria** apply to one function: **payments above an amount need two approvers**. The operational environment exposes the system to **real external threats** and lets you evaluate behaviour **through external interfaces**.
Example	Same password-reset requirement (official Q24). Valid criteria: reset works under the stated conditions · **more than two requests without a reset → ID locked · more than two security questions missed → ID locked · email error → ID locked** · the new password works for login. Invalid: a criterion that **drops "without a reset"** · one that says **"error"** where the requirement says **"locked"** · "prompt the user with the password rules", which is **tempting and logical, but not in the requirement**.
Exam angle	**Filter each candidate criterion:** is every condition stated in the requirement? Is the outcome the **same word** (locked vs error)? Has a qualifier been **dropped**? Is it "logical but unspecified"? If any answer is wrong, **reject** the criterion.
Don't confuse	Acceptance criteria **do not improve** a requirement. Improving it is a requirements-review activity (AS-4.2.1). At acceptance, you hold the system to the requirement **as written**.

Security regression testing in maintenance (AS-4.6.1, K3 pool)	
In plain words	After any change, **regression testing** checks that what used to work **still works**. For security, that means the system **still resists attempts to defeat its controls**, and the change **introduced no new vulnerabilities**.
How it works	**Changes aimed at usability or efficiency are especially likely to weaken security controls.** Regression suites built from **individual function tests are often not enough** for security. **End-to-end security scenarios** give much more confidence that **complete transactions** stay secure. Run the security scenarios **every time anything changes**: **hardware, configuration files, OS, DBMS, network, or software**.
Example	The syllabus's scenarios: **(1)** a user can log in and complete a purchase **without exposing personal information**; **(2)** a user can only perform actions within their rights. For example, a payroll clerk can **add** an employee but **cannot see their bank details**. After a "faster checkout" change, scenario 1 shows the card number now appears in the confirmation URL.
Exam angle	Prefer the approach with **end-to-end security scenarios, run after every kind of change** (not only code changes). Reject approaches that rerun only the changed function's unit tests.
Don't confuse	**Confirmation testing** checks that a **fix** works. **Regression testing** checks that **nothing else** broke, including security controls the change did not touch on purpose.

2.6 Fuzzy points clarified

Places where the wording is easy to misread

Where does system testing run? §4.5.1 says it is **usually in a development environment**. §4.1.2 says it runs **in some approximation of the final target environment**. These fit together: a development-controlled environment built to **resemble** the target. Acceptance testing moves to the **realistic operational** environment.

"Minimum" means two different things. In **EP** (Q22) it means **one value per class**. In **condition lists** (Q23) it means **every stated rule, valid and invalid, with nothing extra**. In both cases the trap is the same: adding tests the question did not ask for, or leaving out the valid case.

The SQL character in Q22 is a plain single quote ('). Printed guides often show it as a curly quote (‘). Both mean the same apostrophe character.

Statement coverage = lines of executable code (the syllabus's wording). **Branch coverage = % of decision outcomes**. Exam options swap them.

The CERT example tests and the IEEE "deficiency it reveals" column in Section 1 are this guide's illustrations. The syllabus lists the practices and principles. Expect questions on the **names**, and K3/K4 scenarios that apply them.

Q18 option a is half right. Security requirements should be **documented** (not hidden) **and** protected from those who do not need to know. An option that says "do not document them formally" is wrong.

2.7 Chapter 4 keyword glossary

Keyword	Working definition for the exam
abuse case	A use-case-like description of how a system could be misused; a source for end-to-end security scenarios and a requirements-based coverage measure (% use/abuse cases tested).
fuzz testing	Inputting **massive amounts of random data (fuzz)** to discover vulnerabilities; white-box fuzzing instruments code to detect memory corruption and buffer overflows.
backdoor	Undocumented interface known only to the developer, intentionally implemented to bypass normal security controls.
cyclomatic complexity	Number of independent paths through an element.
scaffolding	Stubs and drivers used during integration; removed incrementally as real components arrive.

Two keywords, one chapter

The syllabus lists only **abuse case** and **fuzz testing** as Chapter 4 keywords, but the questions lean on the vocabulary of **coverage** and **test levels** from Foundation Level. If any of statement, branch, path, equivalence partitioning or state transition coverage feels shaky, revise it before Section 4.

Section 3 — Tables and Summaries

3.1 Which level finds what?

Level	Typical security focus	Box
Static (reviews, analysis)	Requirements gaps, design flaws, coding-standard violations, malicious code	—
Component	Secure coding practices, input validation, buffer overflow, backdoors, fuzzing	White + black
Integration	New attack vectors from interactions; scaffolding removal reveals paths	White + black
System	End-to-end; emergent properties; transaction security; "x should not allow y"	**Black**
Acceptance	Security acceptance criteria in a realistic environment; external interfaces	**Black**
Maintenance	Confirmation + end-to-end security regression after any change	Black (mainly)

3.2 Coverage measure → testing family

Measure in the stem	Family
% requirements / % use and abuse cases / % critical functions or mission threads	Requirements-based
One value per equivalence class; pairwise; N-wise	Data-driven
Cause-effect; postcondition disjuncts; axioms; states, transitions, transition-pairs, cycles	Model-based
% packages, classes, methods, decisions, lines (statement)	Structural
All entry-to-exit paths (strongest) · critical paths · % decision outcomes (branch)	Structural / control flow

3.3 From a security rule to test conditions (Q23 method)

Rule in the requirement	Valid condition	Invalid condition
User must exist	Valid user	Invalid user
Answer 2 of 3 security questions	2 correct answers	2 incorrect answers
Link sent to email	Good email	Bad email
Reset to a new password	Reset with good password	Reset with bad password
Link disabled after 1 hour	Link good	Link expired
Max two requests without reset	Two requests without reset	Three requests without reset

3.4 Acceptance-criteria filter (Q24 method)

Test for each candidate criterion	Keep if…
Is every condition in it stated in the requirement?	Yes — no invented rules
Is the outcome the requirement's outcome (locked vs error)?	Exactly the same word
Is a qualifier missing ("without a reset")?	No qualifier dropped
Is it "logical but unspecified"?	Then reject it

Reproduce from memory before moving on

Four lifecycle reasons and **phase activities** — 3 points guaranteed (AS-4.1.1, 4.1.2).

Coverage families and the **EP minimum** method — 3 points guaranteed (AS-4.4.3).

Acceptance-criteria filter — 2 points guaranteed (AS-4.5.2).
Requirements checklist and **IEEE ten** — the 3-point K4 pool.
Integration ≠ sum and **condition table** — the K3 pools.

Section 4 — Demo Questions with Answers

*Questions 1–8 are the **official ISTQB CTAL-SEC Sample Exam questions for Chapter 4** — #17 to #24 — with official answers and rationale. LO tags are inferred and match the Exam Structure Table. Questions 9–32 are derived strictly from the syllabus. Correct options are marked ✔.*

Part A — The 8 official Chapter 4 questions

Q1. OFFICIAL · Sample Exam #17 (1 pt)

At what point in the SDLC should there be checking to ensure that proper secure coding practices have been followed?

a) Component testing ✔

b) Integration testing

c) System testing

d) Security acceptance testing

Correct answer: a | LO: AS-4.1.1 (K2) | Syllabus §4.1.1

Why: (a) — official rationale: **the checking should be done as soon as the code is written.** Syllabus §4.1.1 (checkpoints for testing): **in development, component tests should be documented and performed to verify that secure coding practices have been followed and successfully implemented**.

Q2. OFFICIAL · Sample Exam #18 (2 pts)

You have been asked by the business analyst to help with defining the requirements for the security aspects of a system. This is a safety-critical system that stores medical information for patients and supplies this information to health professionals at hospitals, doctors' offices and ambulances. At what point in the lifecycle should the security requirements be documented and at what level of detail?

a) They should not be documented formally because of the need to protect the security implementation within the code from outsiders

b) They should be documented in a detailed and unambiguous way in the requirements documents during the requirements phase ✔

c) They should be documented during the design phase when the code approach is known rather than at the requirements phase when the approach is not known

d) They should be restricted to the functional access and availability requirements from the user's perspective and should be documented during the requirements phase

Correct answer: b | LO: AS-4.1.2 (K3) | Syllabus §4.1.2

Why: (b) — official rationale. (a) is wrong, **although it is important that the documented requirements be protected from those who do not need to know**. (c) **although they may be refined at the design level, they should be initially captured during the requirements definition phase**. (d) **security requirements also need to include secure coding practices, etc.** Syllabus §4.1.2: **documenting security needs in a way that is detailed and unambiguous**, traceable to implementation and testing.

Q3. OFFICIAL · Sample Exam #19 (3 pts)

A deficiency has been discovered in production. If an unauthorized user copies a URL from a session of an authorized user, the unauthorized user can paste the URL into their session and continue to process with the authorized user's rights. In the case that was reported, the unauthorized user was able to use the authorized user's URL to change the system administration password. In order to close this gap, the developers will check the session ID and the user ID anytime a URL is used.

What is a realistic concern for this fix?

a) It will not fix the problem and session hijacking will still be possible

b) It will fix the problem, but the usability may be adversely affected

c) It will fix the problem, but performance may be adversely affected ✔

d) It will not fix the problem and will expose a new vulnerability with session IDs

Correct answer: c | LO: AS-4.3.1 (K4) | Syllabus §4.3, §4.2

Why: (c) — official rationale: **it is likely that this level of checking will slow down the system because it will have to check on each screen change.** (a) and (d) **are not correct because the fix should fix the problem**. (b) **there should be no impact to usability (unless you are the hacker!)**. The design flaw violated **"authorize after you authenticate"**; the requirements checklist names the **security/performance trade-off**.

Q4. OFFICIAL · Sample Exam #20 (1 pt)

During component level testing, why should the security tester review compiler warnings?

a) Because these indicate security problems that must be fixed

b) Because these indicate potential issues that should be investigated ✔

c) Because these indicate coding issues that will cause functional defects

d) Because these indicate poor programming practices that will increase maintainability

Correct answer: b | LO: AS-4.4.1 (K2) | Syllabus §4.4.1, §4.4.2

Why: (b) — official rationale: **from a security testing standpoint, compiler warnings indicate potential issues that could lead to security gaps.** (a) **warnings do not necessarily require a fix**. (c) and (d) **may be true, but are not related to security testing**. CERT practice #2: **heed compiler warnings**.

Q5. OFFICIAL · Sample Exam #21 (2 pts)

You have been testing a system that has 20 defined components. You have done extensive security testing on each of the components. The system is now ready to move into component integration security testing. How should you approach this testing?

a) Since component integration testing is concerned with the summation of the vulnerabilities of the individual components, conducting the same tests on the integrated components is the best approach.

b) The main risk is now in the integration of the components themselves, so testing should cover each interface and verify that there are no vulnerabilities in the interfaces and the components should also be retested.

c) It is likely that new vulnerabilities are present with the integrated components as well as with the larger system and infrastructure that is now testable, so testing should expand to include these new areas. ✔

d) Since the components are now integrated, the security risks will be reduced because the possible interactions are now limited so only the integration points should be tested and no component re-testing is needed.

Correct answer: c | LO: AS-4.4.5 (K3) | Syllabus §4.4.4, §4.4.5

Why: (c) — official rationale: **new vulnerabilities may be present with the integrated components and new testing areas are likely to be available.** (a) **component integration testing is not the sum of the individual components**. (b) **testing should not be limited to just the interfaces and the original components**. (d) **security risks are likely to be increased, not decreased**.

Q6. OFFICIAL · Sample Exam #22 (3 pts)

You are creating security test cases to check for SQL injection on an input field that allows up to 5 alphanumeric characters. You are planning to apply equivalence partitioning to reduce the number of test cases you will need to execute. Given this information, which of the following is the minimum set of inputs you would need to use to test this field?

a) bbbbb, 12345, ‘

b) %, ‘, @, ab123

c) ‘, ab123 ✔

d) ‘

Correct answer: c | LO: AS-4.4.3 (K4) | Syllabus §4.4.3

Why: (c) — official rationale: **this has one test for SQL injection and one for a valid input. This is the minimum number of tests.** (a) and (b) **have more than the minimum**; (d) **doesn’t have enough tests because it doesn’t test the valid input. It would be advisable to do more testing on the various characters that can support SQL injection, but this question is asking to apply EP and get the minimum number of test cases.** Syllabus §4.4.3: **one representative from each class**.

Q7. OFFICIAL · Sample Exam #23 (2 pts)

You have been given the following requirement for security testing. A user will be allowed to request their password. If they make this request, they must answer two of their three security questions correctly. If they answer correctly, a link will be sent to their email. The link will take them to a page where they can reset their password. Once reset, they can login with the new password. The link must be disabled 1 hour after it is sent. The user is allowed only two password requests without a reset, after which they will have to call the help desk. For any other errors, the user ID is locked and must be unlocked by the help desk.

Which of the following is the minimum list of test conditions to adequately test the functional security covered by this requirement?

a) Invalid user; valid user; 2 correct answers; 2 incorrect answers; good email; bad email; reset with good password; reset with bad password; link good; link expired; two requests without reset; three requests without reset ✔

b) Valid user; 2 correct answers; good email; reset with good password; link good; two requests without reset

c) Invalid user; 2 incorrect answers; bad email; reset with bad password; link expired; three requests without reset

d) Buffer overflow on each input field; SQL injection on each input field; XSS on the login page and reset password page, invalid user; valid user; 2 correct answers; 2 incorrect answers; good email; bad email; reset with good password; reset with bad password; link good; link expired; two requests without reset; three requests without reset

Correct answer: a | LO: AS-4.5.1 (K3) | Syllabus §4.5.1

Why: (a) — official rationale: **it covers the main scenarios for the functional security specified in the requirement. B tests only the valid tests. C tests only the error conditions. D expands into attack testing as well as functional testing.**

Q8. OFFICIAL · Sample Exam #24 (2 pts)

A user will be allowed to request their password. If they make this request, they must answer two of their three security questions correctly. If they answer correctly, a link will be sent to their email address. The link will take them to a page where they can reset their password. Once reset, they can login with the new password. The link must be disabled one hour after it is sent. The user is allowed only two password requests without a reset, after which he will have to call the helpdesk. For any other errors, the user ID is locked and must be unlocked by the help desk.

Which of the following is a valid set of acceptance criteria for this requirement?

1. User can reset password if less than three requests have been made since last reset, and two security questions are answered correctly, and the link is used to reset and a valid password is entered at the reset prompt.
2. More than two requests results in user ID locked.
3. More than two requests without a reset results in user ID locked.
4. More than two security questions missed results in error.
5. More than two security questions missed, user ID is locked.
6. If email error is received by the system, user ID is locked.
7. If invalid password is entered on reset, the user is prompted with the proper rules.
8. Reset password can be used to log into the system.

a) 1, 2, 4, 6, 7, 8
b) 1, 2, 3, 4, 5, 6, 7, 8
c) 3, 5, 6, 7, 8
d) 1, 3, 5, 6, 8 ✔

Correct answer: d | LO: AS-4.5.2 (K3) | Syllabus §4.5.2

Why: (d) — official rationale: **it provides the acceptance criteria based on the requirement. 7 is tempting and would be logical, but it is not specified in the requirement. ... 2 is incorrect where 3 is correct. 4 is incorrect where 5 is correct.** Criteria restate the requirement exactly: the qualifier **"without a reset"** and the outcome **"locked"** (for "any other errors") both matter.

Part B — 24 additional syllabus-derived questions

*The K-level in each question's label is the level of the **learning objective** it practises, not of the question itself. Some Part B items are quick recall checks on a K3 or K4 objective. Use them to learn the facts, and use the scenario items (and Part A) to practise the reasoning the real K3/K4 questions need.*

Q9. Derived from syllabus

Which of the following is one of the reasons the syllabus gives for integrating security testing into the software lifecycle?

a) To allow security testing to be outsourced at the end of the project
b) To reduce the number of security requirements
c) To replace secure coding with penetration testing
d) To provide entry and exit criteria throughout the project ✔

Correct answer: d | LO: AS-4.1.1 (K2) | Syllabus §4.1.1

Why: (d). The four reasons: **a prescribed time for security-related activities · checkpoints for review · checkpoints for testing · entry and exit criteria throughout the project** (e.g. no component enters integration until all security activities are complete).

Q10. Derived from syllabus

Why does the syllabus recommend reviewing code changes as a lifecycle checkpoint?

a) To detect the presence of malicious coding by internal employees or contractors ✔
b) To measure developer productivity
c) To check that code comments are complete
d) To decide which tests can be skipped

Correct answer: a | LO: AS-4.1.1 (K2) | Syllabus §4.1.1

Why: (a) — §4.1.1: **code changes should also be reviewed to detect the presence of malicious coding by internal employees or contractors.**

Q11. Derived from syllabus

During which phase are security tests performed in "some approximation of the final target environment", requiring a transition from the development environment?

a) Component testing

b) System testing ✔

c) Requirements review

d) Implementation

Correct answer: b | LO: AS-4.1.2 (K3) | Syllabus §4.1.2

Why: (b) — §4.1.2 system testing activities: **performing security tests in some approximation of the final target environment, necessitating a transition from the development environment in which previous implementation and integration activities have occurred**.

Q12. Derived from syllabus

A team is adopting two-week sprints for a banking app. The product owner asks where security activities fit.

Which plan best implements the syllabus's lifecycle activities for this iterative project?

a) Run all security testing in a dedicated hardening sprint before go-live

b) Document security requirements once at project start and never revisit them

c) Capture security needs in each sprint's user stories and acceptance criteria and review them; design on a whiteboard to meet those needs; write secure code with component tests and reviews in-sprint; run system-level security tests frequently in an approximation of the target environment; regression-test security after each change ✔

d) Leave security to acceptance testers, because acceptance testing is black-box

Correct answer: c | LO: AS-4.1.2 (K3) | Syllabus §4.1.2

Why: (c) maps each syllabus phase into the iteration: requirements **documented, reviewed and traceable**; design **documented appropriately — e.g. on a whiteboard in an iterative approach**; implementation with **component testing and component reviews**; **iterative approaches can system-test much more frequently**; maintenance/regression after change. (a) and (d) defer security to the end.

Q13. Derived from syllabus

A database is upgraded to a new major version on a live system. What does the syllabus say about security testing this change?

a) Maintenance includes upgrades such as databases and should be treated with the same care as original development: confirmation and regression testing to ensure no new vulnerabilities and that existing defenses are still effective ✔

b) Upgrades of third-party components are out of scope for security testing

c) Only a functional smoke test is needed after an upgrade

d) Security testing is only required if application code changes

Correct answer: a | LO: AS-4.1.2 (K3) | Syllabus §4.1.2

Why: (a). §4.1.2: **maintenance can include upgrades (e.g. operating system, databases), coding changes, data conversions and platform migrations ... any maintenance activity should be treated with the same care and attention as the original development.**

Q14. Derived from syllabus

Requirement REQ-17: "The patient portal shall be secure. Access to lab results is only granted to authorized personnel. Patients can view their own results online."

Which deficiency should a security tester raise FIRST?

a) The requirement does not specify the font size for lab results

b) Terms such as "shall be secure" and "authorized personnel" are ambiguous and must be clarified into specific, testable statements (who is authorized, for which data types, under what conditions) ✔

c) The requirement should specify the encryption algorithm to be used

d) The requirement should be moved to the design document

Correct answer: b | LO: AS-4.2.1 (K4) | Syllabus §4.2

Why: (b). The checklist's **testability** heading names exactly these examples: **"processing must be secure" and "access is only granted to authorized personnel"** must be **identified and clarified to be specific and testable**. Until they are, no security test can be written. (c) is a design choice, not a requirement deficiency.

Q15. Derived from syllabus

A requirement for an online bank states: "After three failed logins the account is locked permanently. Unlocking requires a visit to a branch with photo ID." The bank has 2 million mostly online customers, and predictable customer numbers are used as user IDs.

Which security-checklist headings reveal the most significant deficiencies?

a) Performance only — branch visits take time

b) Compliance only — the policy reference is missing

c) Privacy only — photo ID is personal data

d) Usability (no practical remedy for legitimate locked-out users) and common vulnerabilities (predictable IDs plus permanent lockout enable a logical denial of service) ✔

Correct answer: d | LO: AS-4.2.1 (K4) | Syllabus §4.2

Why: (d). **Usability: are remedies specified for legitimate users who may experience problems?** — a branch visit for 2 million online users is not practical. **Common vulnerabilities:** predictable user names + permanent lockout is the syllabus's **logical DoS** example (Chapter 3). These are higher-risk than (a)–(c).

Q16. Derived from syllabus

Which statement about requirements is NOT among the syllabus's general considerations?

a) Requirements are stable once approved and seldom change ✔

b) Many organizations are challenged to write clear, unambiguous, complete, correct and testable requirements

c) Requirements can contain gaps and errors, so both verification and validation are needed

d) Quality characteristics such as security are often overlooked in favor of functionality

Correct answer: a | LO: AS-4.2.1 (K4) | Syllabus §4.2

Why: (a) contradicts §4.2: **requirements are highly subject to change throughout a project**.

Q17. Derived from syllabus

Design extract: "To improve responsiveness, the mobile client validates all input and checks the user's role before calling the REST API. The API trusts requests that include a valid session token and executes the requested operation. Reports are generated by concatenating user-supplied filter text into a SQL statement."

Which IEEE Center for Secure Design principles does this design violate?

a) Always consider the users; be flexible about future changes

b) Use cryptography correctly; identify sensitive data

c) Earn or give, but never assume, trust (server trusts client validation and role checks); authorize after you authenticate (the API does not authorize the operation); strictly separate data and control instructions (user text concatenated into SQL) ✔

d) Understand how external components change the attack surface

Correct answer: c | LO: AS-4.3.1 (K4) | Syllabus §4.3

Why: (c). Client-side validation and role checks **assume trust** in the client; the API authenticates (token) but **does not authorize** the operation; concatenating user data into SQL **mixes data and control instructions** from an untrusted source. Each maps to a named IEEE approach.

Q18. Derived from syllabus

A design adds a third-party analytics SDK and a new partner API to an existing application without updating the threat model. Which IEEE design principle is most directly at stake?

a) Authorize after you authenticate

b) Understand how integrating external components changes your attack surface ✔

c) Use an authentication mechanism that cannot be bypassed

d) Always consider the users

Correct answer: b | LO: AS-4.3.1 (K4) | Syllabus §4.3

Why: (b) — the IEEE list includes **understand how integrating external components change your attack surface**.

Q19. Derived from syllabus

Which vulnerability is specifically named by the syllabus as identifiable and fixable during structural testing?

a) Social engineering

b) Weak password policy

c) "Backdoor" access — an undocumented interface known only to the developer that bypasses normal security controls ✔

d) Denial of service from a botnet

Correct answer: c | LO: AS-4.4.1 (K2) | Syllabus §4.4.1

Why: (c). §4.4.1.1 lists **memory buffer overflows**, **malicious code inserted by an internal employee or contractor**, and **"backdoor" access**.

Q20. Derived from syllabus

What advantage does white-box fuzz testing have over black-box fuzzing, according to the syllabus?

a) On small software blocks (functions, classes) it might get usable results in much less time, and by instrumenting code it can detect memory corruption and buffer overflow ✔

b) It needs no random data

c) It replaces the need for functional security testing

d) It is the only way to test encryption

Correct answer: a | LO: AS-4.4.1 (K2) | Syllabus §4.4.1

Why: (a) — §4.4.1.1: **white-box fuzz testing (on small software blocks, functions, classes) might get usable results in much less time than a black-box fuzzing tool**; **white-box fuzz testing tools are able to detect memory corruption, buffer overflow, etc., by instrumenting the code**.

Q21. Derived from syllabus

How is the adequacy of functional security testing at any level determined?

a) By the number of tests executed per day

b) Only by observing responses to unspecified stresses

c) By the absence of compiler warnings

d) By confirming satisfaction of specified security requirements, in addition to observing responses to stresses not explicitly specified — which requires creativity ✔

Correct answer: d | LO: AS-4.4.1 (K2) | Syllabus §4.4.1.2

Why: (d) — §4.4.1.2, which adds that **creativity is required … because testers are probing for what the software developers have overlooked**.

Q22. Derived from syllabus

Coding specification for a component that writes customer notes to a database and forwards them to a CRM: "Notes up to 500 characters; only users with role AGENT may write; the component runs under a dedicated service account; outbound data to the CRM must be encoded."

Which set of abstract component-level security tests best reflects the CERT practices?

a) Measure throughput of 1,000 writes per second

b) Submit notes over 500 characters and with SQL metacharacters (validate input); attempt a write as a non-AGENT user (default deny / policy); verify the service account has only the rights needed (least privilege); send notes with control characters and verify encoding before the CRM call (sanitize data sent to other systems) ✔

c) Verify the user interface colors meet branding guidelines

d) Run the component with administrator rights to check it works

Correct answer: b | LO: AS-4.4.2 (K3) | Syllabus §4.4.2

Why: (b) maps the specification to CERT practices **validate input, default deny, architect for security policies, least privilege, sanitize data sent to other systems**. (d) contradicts least privilege. Tests should focus on **the most crucial requirements in terms of probability of attack and consequences of compromise**.

Q23. Derived from syllabus

On what basis should component tests against secure-coding checklists be focused?

a) A well-documented risk analysis incorporating realistic threat modeling — focusing on the most crucial requirements in terms of probability of attack and consequences of compromise ✔

b) Equal effort on all ten practices regardless of risk

c) The developer's preference

d) The number of lines of code per component

Correct answer: a | LO: AS-4.4.2 (K3) | Syllabus §4.4.2

Why: (a) — §4.4.2.

Q24. Derived from syllabus

Component test results for an authorization module: statement coverage 100%; branch coverage 62%; all 14 requirements tested; 3 of 9 abuse cases tested. The untested branches are all in the code path that handles expired sessions.

What is the best assessment of the adequacy of the code from a security perspective?

a) Adequate — 100% statement coverage proves all code works

b) Adequate — all requirements are tested

c) Not yet adequate — branch coverage and abuse-case coverage are low, and the untested decision outcomes are in security-relevant expired-session handling; tests should target those outcomes and the remaining abuse cases ✔

d) Not adequate — path coverage must be 100% before release

Correct answer: c | LO: AS-4.4.3 (K4) | Syllabus §4.4.3

Why: (c). Structural coverage: statement coverage is weaker than **branch coverage (% of decision outcomes)**; untested outcomes lie in **security-relevant** code. Requirements-based coverage includes **% of specified use/abuse cases tested** — only 3 of 9. (d) is unrealistic: **exhaustive path testing is generally not feasible because of loops**.

Q25. Derived from syllabus

Which is the strongest of the control-flow-based coverage criteria named in the syllabus?

a) Statement coverage

b) Branch coverage

c) Critical path coverage

d) Path coverage ✔

Correct answer: d | LO: AS-4.4.3 (K4) | Syllabus §4.4.3

Why: (d) — **path coverage, which measures against all entry-to-exit paths**; **generally not feasible** because of loops, so **critical path** and **branch** coverage are less stringent alternatives.

Q26. Derived from syllabus

A state-transition model of a session (Anonymous → Authenticated → Elevated → Expired) is used for security testing. Which coverage criteria belong to this kind of model?

a) Percentage of requirements and use cases tested

b) Percentage of states, transitions, transition-pairs and cycles ✔

c) Coverage of axioms

d) Percentage of packages and classes executed

Correct answer: b | LO: AS-4.4.3 (K4) | Syllabus §4.4.3

Why: (b) — for **transition-based models** the syllabus lists **% nodes (states), % transitions, % transition-pairs, % cycles**. (c) is for **algebraic** notations; (a) is requirements-based; (d) structural.

Q27. Derived from syllabus

A numeric PIN field accepts exactly 4 digits. Applying equivalence partitioning with the aim of a minimum set for security-relevant input validation, which set is best?

a) 1234 (valid), 12a4 (non-numeric), 123 (too short), 12345 (too long) ✔

b) 0000, 1111, 2222, 3333

c) 1234 only

d) ‘ only

Correct answer: a | LO: AS-4.4.3 (K4) | Syllabus §4.4.3

Why: (a) gives **one representative per class**: valid; invalid type (non-numeric); invalid length **below** the range; invalid length **above** the range. “Too short” and “too long” are **two separate invalid partitions**, because the system can handle each side differently. (b) repeats one class; (c) and (d) miss classes — just as in the official Q22, a set without **a valid input** is incomplete.

Q28. Derived from syllabus

Which statement about security during component integration testing is correct?

a) Vulnerabilities after integration equal the sum of the vulnerabilities of each component

b) Integration can only increase security

c) New attack vectors become possible through interactions, while some interactions may mitigate or block breach sequences; the integration approach (top-down, bottom-up) can affect when concerns are revealed ✔

d) Integration testing does not require creativity because the interfaces are specified

Correct answer: c | LO: AS-4.4.4 (K2) | Syllabus §4.4.4

Why: (c) — §4.4.4 in full; the syllabus also says testers **need to be creative** at this level.

Q29. Derived from syllabus

During integration, stubs replace the payment and identity services. As the real services are integrated and stubs removed, what should the security tester expect and do?

a) Expect fewer vulnerabilities and reduce testing

b) Stop testing, since the services were tested by their vendors

c) Rerun only the original component tests

d) Expect new paths to vulnerabilities to become exploitable and design additional integration security tests, based on risk analysis and threat modeling ✔

Correct answer: d | LO: AS-4.4.5 (K3) | Syllabus §4.4.5

Why: (d) — §4.4.5: **as more implemented components are added ... scaffolding is incrementally removed, allowing for fuller assessment of functionality as well as new paths to vulnerabilities that might be exploited.**

Q30. Derived from syllabus

Security requirement: "While transferring funds, the system shall not allow the debit account to be changed after the one-time code has been verified." Functional process: select accounts → enter amount → receive one-time code → verify code → confirm transfer.

Which end-to-end security test scenario best verifies the requirement within the functional process?

a) Transfer funds between two accounts and check the balance

b) Perform the transfer up to code verification, then attempt to change the debit account (via UI and by tampering with the request) before confirmation; verify the change is rejected and the transfer uses the originally verified account, and that the attempt is logged ✔

c) Run a vulnerability scanner against the transfer page

d) Verify that the one-time code is six digits

Correct answer: b | LO: AS-4.5.1 (K3) | Syllabus §4.5.1

Why: (b) follows the syllabus pattern **"in the process of doing x the system should not allow y to happen"** — it runs the **functional process end to end** while **probing for ways the security constraint might be violated**.

Q31. Derived from syllabus

When is the best time to define and document acceptance criteria, according to the syllabus?

a) Before system development or purchase ✔

b) After system testing is complete

c) During deployment

d) After the first production incident

Correct answer: a | LO: AS-4.5.2 (K3) | Syllabus §4.5.2

Why: (a) — §4.5.2, so **an initial understanding can be made between the supplier and acquirer, even if both are in the same**

organization; changing criteria should be **analyzed for their impact on security testing**.

Q32. Derived from syllabus

A retailer releases a change that speeds up checkout by caching customer profiles. The existing regression suite contains 600 individual function tests.

Which security regression approach best follows the syllabus?

a) Rerun only the 600 function tests

b) Skip security regression — performance changes do not affect security

c) Run a defined set of end-to-end security scenarios (e.g. log in and complete a purchase without compromising personal information; users only perform actions in their rights), paying particular attention to the caching change because efficiency enhancements are prone to sacrificing security controls ✔

d) Run a penetration test once a year

Correct answer: c | LO: AS-4.6.1 (K3) | Syllabus §4.6

Why: (c) — §4.6: **enhancements to usability or efficiency are especially prone to sacrificing security controls**; individual function tests are **often insufficient**; **end-to-end regression test scenarios are more robust** and should be run **each time a change is made**.

Most Important Questions From This Chapter

*Ranking rationale: guaranteed **AS-4.1.1 (K2), 4.1.2 (K3), 4.4.3 (K4), 4.5.2 (K3)**; pools 4.2.1/4.3.1 (K4), 4.4.1/4.4.4 (K2), 4.4.2/4.4.5 (K3), 4.5.1/4.6.1 (K3).*

Rank	Topic / question focus	LO	Why it matters
1	Coverage analysis and EP minimum — Q6, Q24–Q27	AS-4.4.3	**Guaranteed 3 points.**
2	Requirements and design deficiencies — Q3, Q14–Q18	AS-4.2.1 / 4.3.1	**3 points** from the pool; checklist + IEEE ten.
3	Acceptance criteria — Q8, Q31	AS-4.5.2	**Guaranteed 2 points**; no invented rules.
4	Lifecycle activities — Q2, Q11–Q13	AS-4.1.2	**Guaranteed 2 points.**
5	End-to-end scenarios and regression — Q7, Q30, Q32	AS-4.5.1 / 4.6.1	**2 points** from the pool.
6	Component and integration tests — Q4, Q5, Q19–Q23, Q28, Q29	AS-4.4.x	**1 + 2 points** from two pools.
7	Why a lifecycle — Q1, Q9, Q10	AS-4.1.1	**Guaranteed 1 point.**

Section 5 — Exam Tricks

5.1 Frequently tested topics (evidence-based)

LO	K	Qs	Pts	Sample #	Format used officially
AS-4.1.1	K2	1	1	#17	"At what point in the SDLC…" → **component testing**
AS-4.1.2	**K3**	1	2	#18	When and how to document security requirements
AS-4.2.1 / 4.3.1	**K4**	1	**3**	#19	Production deficiency + proposed fix → **realistic concern**
AS-4.4.1 / 4.4.4	K2	1	1	#20	Why review compiler warnings?
AS-4.4.2 / 4.4.5	**K3**	1	2	#21	How to approach integration security testing
AS-4.4.3	**K4**	1	**3**	#22	EP minimum input set for SQLi
AS-4.5.1 / 4.6.1	**K3**	1	2	#23	Minimum condition list for a requirement
AS-4.5.2	**K3**	1	2	#24	Pick the valid acceptance criteria from 8
Chapter 4 total	—	**8**	**16**	**#17–#24**	**20% of the paper — the largest chapter**

The arithmetic

Chapter 4 is **16 of 80 points**. Its four K3 questions are worth **8 points**, and **two of them share one requirement** (password reset). Master the **condition table** and the **acceptance-criteria filter** on that one requirement and you have answered **4 points** of the official paper.

5.2 Common distractors observed

- **Over-testing** — adding buffer overflow, SQLi and XSS when the question asks for **functional security** conditions; adding extra characters when it asks for the **EP minimum**.
- **Under-testing** — valid conditions only, or invalid only, or an attack input without a valid input.
- **"Logical but unspecified"** acceptance criteria — e.g. prompting the password rules on a failed reset.
- **Dropped qualifiers** — "more than two requests" vs "more than two requests **without a reset**"; "error" vs "**locked**".
- **Integration as a sum** of component vulnerabilities, or as **reducing** risk.
- **Security requirements in design, or undocumented** for secrecy.
- **Warnings that "must be fixed"** — they are **potential issues to investigate**.
- **A fix that "won't work"** when it clearly closes the gap — look instead for the **trade-off** (performance).

5.3 Keyword recognition (map words → answers)

If the stem says…	Think…
"secure coding practices followed"	**Component testing**
"security requirements documented"	**Requirements phase, detailed and unambiguous**
"secure", "authorized personnel" in a requirement	**Testability deficiency** — make specific
"check on every request / every URL"	**Performance trade-off**
"compiler warnings"	**Potential issues to investigate**
"20 components tested; now integrate"	**New vulnerabilities; expand testing** to the larger system
"stubs and drivers removed"	**New paths to vulnerabilities**
"minimum", "equivalence partitioning"	**One per class** incl. a **valid** value

If the stem says...	Think...
“statement coverage 100%”	Weaker than branch; check **decision outcomes**
“all entry-to-exit paths”	**Path coverage** (strongest, usually infeasible)
“abuse cases”	Requirements-based coverage / end-to-end scenario source
“doing x should not allow y”	**System-level** security scenario
“acceptance criteria”	**Restate the requirement exactly**
“efficiency / usability enhancement”	**Prone to sacrificing security controls** → regression

5.4 Typical traps

- **System vs acceptance.** System = first end-to-end, usually **development** environment, emergent properties. Acceptance = **realistic operational** environment, external interfaces and **real threat agents**.
- **Both are black-box.** Component and integration testing exploit internals.
- **Requirements-based vs data-driven vs model-based vs structural** coverage — classify the measure, not the tool.
- **Algebraic models → axioms**; **pre-post → cause-effect and postcondition disjuncts**; **transition models → states, transitions, pairs, cycles**.
- **Fuzzing** is random data in **massive amounts**; white-box fuzzing is **faster on small blocks** and **instruments the code**.
- **Maintenance types**: corrective (fix), adaptive (environment change), perfective (enhance).

5.5 How to eliminate wrong answers

6. For **minimum sets**, strike any set lacking a **valid** case, then any with **duplicates of a class**.
7. For **condition lists**, strike lists with only one polarity, then lists that add **attack** tests the stem did not request.
8. For **acceptance criteria**, compare each number word-for-word with the requirement; strike sets containing an unspecified or mis-worded criterion.
9. For **fix analysis**, if the fix plainly addresses the cause, strike “won’t fix”; then choose the **trade-off** the syllabus names (security vs performance or usability) that applies.
10. For **lifecycle timing**, choose the **earliest** phase where the activity is possible.

5.6 Scenario-solving shortcuts

Scenario shape	Shortcut
Requirement text with rules and limits	Build the **valid/invalid table** (3.3)
Same requirement, “acceptance criteria”	Apply the **filter** (3.4)
Input field, “EP minimum”	**One valid + one per invalid class asked about**
Coverage numbers	Find the **low** family and **where** the gaps are
Design extract	Run the **IEEE ten**; trust, authorize-after-authenticate, data/control separation are the usual hits
Requirement extract	Run the **six checklist headings**; testability first
Change to a live system	**End-to-end security regression**, especially for efficiency/usability changes

5.7 Common candidate mistakes

- Applying “more testing is better” to **minimum** questions.
- Applying “better requirements” instincts to **acceptance-criteria** questions — criteria **must not improve** on the requirement.

- Forgetting that **coverage is the key adequacy measure** at component level.
- Treating CERT's ten practices as optional; one or two appear in every component question.

5.8 Mnemonics that carry the whole chapter

Mnemonic	Expands to
T-R-T-E ("Try Each Time")	Lifecycle reasons: prescribed **T**ime, **R**eview checkpoints, **T**est checkpoints, **E**ntry/exit criteria
P-C-V-T-U-P ("Please Check Very Thoroughly, Users Pay")	Requirements checklist: **P**rivacy, **C**ompliance, **V**ulnerabilities, **T**estability, **U**sability, **P**erformance
R-D-M-S	Coverage families: **R**equirements-based, **D**ata-driven, **M**odel-based, **S**tructural
"Sum? No — Some new"	Integration is not the sum of component vulnerabilities; new attack vectors appear
"Valid + Invalid, nothing invented"	K3 design questions in one line
"Faster = Weaker"	Usability/efficiency enhancements are prone to sacrificing security

5.9 Chapter 4 mind map

Branch	Leaves
Lifecycle	Built in, not patched · four reasons · phase objectives and activities · black-box system/acceptance
Requirements	Five truths · six-heading checklist · ambiguous terms
Design	Security-degrading practices · IEEE ten
Component	Static then dynamic · white-box · fuzzing · overflows, malicious code, backdoors · creativity · CERT ten · coverage families · control-flow criteria
Integration	Not a sum · mitigation possible · top-down/bottom-up timing · scaffolding removal
System / acceptance	Emergent properties · x-not-y · use/abuse/process/state models · realistic environment · criteria early · global vs specific
Maintenance	Resist defeat · usability/efficiency risk · end-to-end security regression · all change types

Section 6 — Revision Notes

6.1 One-page chapter summary

Lifecycle. Security is built in through design and verification, not patched in. Integrating testing gives prescribed times, review and test checkpoints, and entry/exit criteria. Each phase has security objectives and activities: requirements (all perspectives, detailed and unambiguous), design (meet them), implementation (secure code, component tests and reviews), system (end-to-end in an approximate target environment), acceptance (users vs criteria in the operational environment), maintenance (confirmation and regression after any change).

Requirements and design. Analyze requirements with the six-heading checklist — privacy, compliance, vulnerabilities, testability, usability, performance; vague words are the first deficiency. Analyze designs against the IEEE ten; trust, authorization after authentication, and data/control separation are the usual failures.

Component and integration. Static, then dynamic. White-box targets controls and measures coverage; fuzzing finds memory faults; structural testing finds overflows, malicious code and backdoors. Design component tests from CERT's ten practices, focused by risk. Adequacy is judged by coverage — requirements-based, data-driven, model-based, structural. Integration creates new attack vectors; removing scaffolding exposes new paths.

System, acceptance, maintenance. System tests probe "doing x should not allow y" with scenarios from use, abuse, process and state models. Acceptance criteria are defined early and restate the requirement precisely. After any change, run end-to-end security regression scenarios — especially after efficiency or usability improvements.

6.2 Key definitions to memorize

Term	Definition
Fuzz testing	Massive amounts of random data input to discover vulnerabilities
Backdoor	Undocumented developer-only interface bypassing security controls
Abuse case	Description of misuse; source for scenarios and coverage
Path coverage	All entry-to-exit paths — strongest, usually infeasible
Branch coverage	Percentage of decision outcomes exercised
Statement coverage	Percentage of executable lines exercised

6.3 High-priority concepts (study first)

11. Coverage families and EP minimum (**AS-4.4.3**, 3 pts).
12. Requirements checklist and IEEE ten (**AS-4.2.1 / 4.3.1**, 3 pts).
13. Acceptance-criteria filter (**AS-4.5.2**, 2 pts) and condition table (**AS-4.5.1**, 2 pts).
14. Lifecycle activities (**AS-4.1.2**, 2 pts) and reasons (**AS-4.1.1**, 1 pt).
15. Integration not a sum; scaffolding; CERT ten (**AS-4.4.x**, 1 + 2 pts).

6.4 Last-minute revision sheet

If you read nothing else

Secure coding checked at **component testing** — as soon as code is written.
Security requirements: **requirements phase, detailed and unambiguous**.
Vague requirement words → **testability** deficiency. Branch-visit-only unlock → **usability**.
Session/URL check on every request → fixes it, **costs performance**.
Compiler warnings → **potential issues to investigate**.
Integration → **new vulnerabilities; expand testing**.
EP minimum → **one valid + one invalid per class** (‘ and ab123).

Conditions → **valid and invalid for every rule, no attacks unless asked**.
Acceptance criteria → **exact restatement**; "without a reset", "locked"; nothing logical-but-unspecified.
Efficiency changes → **end-to-end security regression**.

6.5 Memorization checklist

✓	I can...
☐	give the four lifecycle reasons with an example each
☐	state objectives and activities for all six phases
☐	list the six requirements checklist headings with a question each
☐	name eight of the ten IEEE design approaches
☐	name the three structural-testing vulnerabilities and define a backdoor
☐	recite the CERT ten
☐	classify any coverage measure into its family
☐	build a valid/invalid condition table from a requirement
☐	filter candidate acceptance criteria
☐	design an end-to-end security regression set

6.6 Exam-day quick review

- Largest chapter: **16 points**. Expect **two scenario questions on one requirement**.
- **Minimum** means minimum. **Acceptance criteria** mean the requirement, exactly.
- **Integration adds; it does not sum.**